

Running the app, and proving it works

How to bring the stack up after a month away, who to log in as, what every route does, and three scripted walkthroughs that exercise the coach → client commentary loop end to end.

Your database was 9 migrations behind the code – this has been fixed.

The core “client reads the coach’s commentary” request was returning *HTTP 500* because the app ran STEP-13 code against a STEP-10 schema. All 9 pending migrations have been applied and the loop is now verified working. Details in §1.2.

OPEN THIS URL

<https://app.speechcoach.test>
(never localhost:5173)

EVERY PASSWORD

5 seeded accounts

VERIFIED

30 September 2026
against the running stack

0 What you are holding

Every factual claim in this guide was checked against the running application on 30 September 2026 – not against the planning documents, which describe intent and in several places no longer match what was built.

The headline: the product works. Upload → invite → accept → annotate → publish → read back is implemented and was driven end-to-end during this review. Captions and voice transcription are the two features that will not work today, for one concrete reason each.

- 1

Start here

State of the project, the migration fix, the one URL that works
- 2

Containers

What each service does, which are required, cold-start sequence
- 3

Accounts & passwords

Five seeded logins, all verified; plus your own role-less accounts
- 4

How the product works

The access model, the invitation loop, the draft/publish rule
- 5

Test script A – the coach

Review a speech and leave commentary
- 6

Test script B – the client

Upload a speech and invite a reviewer
- 7

Test script C – the read-back

Log in later and see what the coach said
- 8

Route reference

104 API routes and 22 SPA routes, grouped
- 9

Roles & capabilities

Runtime-verified matrix of who can do what
- 10

Known broken edges

What will visibly fail, and why
- 11

Troubleshooting

The traps that cost hours, with their fixes

How claims here were verified

CLAIM TYPE	METHOD
Credentials	Real <code>POST /login</code> through nginx for all five accounts, then an authenticated <code>GET /api/me</code> to prove the session is genuine
Routes	<code>php artisan route:list --json</code> inside the running container, diffed against <code>api/routes/*.php</code> on disk – 104 routes, set-identical
Permissions	<code>Gate::forUser()->allows()</code> executed per role against real fixture rows, not inferred from reading policies
The core loop	Driven over HTTP: coach created a draft, speaker saw 0 notes, coach published, speaker saw 1 note, then cleanup
Video playback	Presigned URL requested as the speaker, then a range request to SeaweedFS returning <code>206 Partial Content</code> , <code>video/mp4</code>
Database state	Direct SQL against the live Postgres volume

1 Start here

Three things changed while you were away, and one of them was breaking the product’s central feature.

1.1 Where the project stands

Steps 00 through 13 are built and committed. Step 14 (deploy hardening) and Step 15 (accessibility) are not started. The last commit is `4f7dd12`, the STEP-13 retrospective, on branch `development`.

STEP	CAPABILITY	STATUS TODAY
00-04	Upload, transcode, play a video	WORKING verified: real 65.7s video streams
05	Invite a reviewer; nobody else can watch	WORKING access model verified
06-07	Write and publish timed commentary	WORKING full loop driven over HTTP
08	The long-form essay review	WORKING separate publish action
09	Captions from Whisper	BLOCKED model file absent – \$10
10	Voice annotation	PARTIAL records & plays; transcript needs Whisper
11	Report, export, delete account	WORKING after the migration fix
12	Admin portal, coach applications	WORKING after the migration fix
13	Connections / social timeline	WORKING after the migration fix
14-15	Deploy hardening, accessibility	NOT BUILT

1.2 The migration gap – found, and fixed

This was breaking the product’s central feature

The `app` container was running current code against a database whose newest migration was dated **18 August**. 39 migration files existed on disk; only **30** had been applied.

The symptom was not subtle. `AnnotationController` reads `$review->reviewer->anonymized_at` – a column added by one of the unapplied migrations. So every request by a speaker to read their coach’s commentary returned **HTTP 500**:

```
MissingAttributeException: The attribute [anonymized_at] either does not
exist or was not retrieved for model [App\Models\User].
at app/Http/Controllers/Api/AnnotationController.php:68
```

These nine migrations were pending and have now been applied:

MIGRATION	STEP	WHAT IT UNBLOCKS
<code>create_reports_table</code>	11	Report a speech

MIGRATION	STEP	WHAT IT UNLOCKS
<code>create_audit_log_table</code>	11	Admin audit trail
<code>create_data_exports_table</code>	11	Export your data
<code>add_anonymized_at_to_users_table</code>	11	Reading any commentary
<code>create_coach_applications_table</code>	12	Apply to be a coach
<code>create_application_documents_table</code>	12	Upload certification PDFs
<code>add_suspended_at_to_users_table</code>	12	Admin suspend a user
<code>create_connections_table</code>	13	Connections & social timeline
<code>add_timeline_indexes_to_reviews_table</code>	13	Profile timeline performance

Verified fixed

All nine applied cleanly (pure `CREATE TABLE` / `ADD COLUMN` / `CREATE INDEX` – additive, no data loss). The failing request was then re-run and returned **HTTP 200** with the coach's three published notes. A database dump was taken beforehand and is preserved in this session's scratchpad.

1.3 The one URL that works

This stack exposes three origins. Only one of them can log you in, and the other two fail in ways that look like application bugs.

URL	VERDICT	WHAT ACTUALLY HAPPENS
<code>https://app.speechcoach.test</code>	USE THIS	nginx reverse-proxies to your host Vite dev server on :5173. Serves live source from <code>web/src</code> – no rebuild needed to see frontend changes. Cookies, CORS and Sanctum are all configured for this hostname and only this hostname.
<code>http://localhost:5173</code>	LOGIN FAILS	Vite serves the page fine (HTTP 200), so it looks like it works – until you log in. This origin is not in <code>CORS_ALLOWED_ORIGINS</code> , so the browser blocks the call to the API and the form shows a generic “ <i>Something went wrong</i> ” banner. This is the error you hit.
<code>http://localhost:8080</code>	LOGIN FAILS	Serves the <i>baked production build</i> from the <code>web</code> image, which is ~22 commits stale. Worse, session cookies come back <code>domain=.speechcoach.test; secure</code> , which a browser on plain-HTTP localhost will not store. Public pages load; auth cannot.
<code>https://api.speechcoach.test</code>	BACKEND	The API origin the SPA calls. Also the only working address for the admin portal: <code>/control-panel</code> .

Why your login failed

You were on `http://localhost:5173`. The backend is pinned to a single origin – `CORS_ALLOWED_ORIGINS=https://app.speechcoach.test`, `SANCTUM_STATEFUL_DOMAINS=app.speechcoach.test`, `SESSION_DOMAIN=.speechcoach.test`. A request from `localhost:5173` comes back stamped with the `app.speechcoach.test` allow-origin header, the browser sees a mismatch and blocks it, and the fetch rejects with no parseable body. The SPA's error helper falls through to its generic message because there is no `{message, errors}` payload to read. Nothing is wrong with your credentials.

2 Containers

Eleven services start by default; two more are profile-gated. The frontend is deliberately *not* containerized – you run Vite yourself on the host.

2.1 What each service is for

SERVICE	CORE FLOW	PORT	PURPOSE
<code>postgres</code>	REQUIRED	5433→5432	Postgres 17. All application data, plus sessions and cache.
<code>valkey</code>	REQUIRED	internal	Redis-compatible queue broker. Every background job flows through it.
<code>seaweedfs</code>	REQUIRED	8333	S3-compatible object store for video, audio and posters. Currently holds 147 MB.
<code>app</code>	REQUIRED	internal 9000	php-fpm running Laravel. Code is baked into the image – see §11.
<code>web</code>	REQUIRED	443, 8080→80	nginx. Terminates TLS for both hostnames, speaks fastcgi to <code>app</code> , and proxies <code>app.speechcoach.test</code> to your host Vite server.
<code>mailpit</code>	REQUIRED	8025	SMTP sink. Where verification and invitation emails land. <code>app</code> will not boot without it.
<code>clamav</code>	REQUIRED	internal 3310	Only scans coach-application PDFs – it touches <i>nothing</i> in the speech path. But <code>app</code> and <code>queue-worker</code> both hard-gate on its health, so the stack will not start without it. Costs ~956 MB RAM, the heaviest container.
<code>queue-worker</code>	REQUIRED	–	<code>default</code> queue: mail, notifications, document scans.
<code>ffmpeg-worker</code>	REQUIRED	–	<code>transcode</code> queue: video transcode, poster frames, voice-note normalization. Needed for any new upload.
<code>scheduler</code>	OPTIONAL	–	Four nightly maintenance commands. Safe to skip for functional testing.
<code>whisper-worker</code>	OPTIONAL	–	<code>captions</code> queue. Non-functional – the model volume is empty. Reserves 2 CPUs / 4 GB when it does run.

Two further services are profile-gated and do not start normally: `whisper-model-init` (downloads the ~142 MiB speech model) and `whisper-smoke` (a test harness). The separate `compose.e2e.yaml` and `compose.whisper-smoke.yaml` files are override files driven by scripts in `scripts/` – you need neither to run the app, and you must never run the e2e stack at the same time as the dev stack because they contend for ports 443, 5433 and 8333.

2.2 Cold start, in order

Idle memory for the full ten-container stack is about **1.4 GB**.

```
# 0. From the repo root
cd /Users/marscheung/SideProjects/Instruction-Speeches-Library

# 1. Rebuild. api/ is BAKED into the image – there is no bind mount,
#    so code on disk is invisible to the container until you do this.
docker compose build app queue-worker scheduler web

# 2. Infrastructure first. clamav is slow on a cold signature volume
```

```
# (start_period is 180s) and everything else waits on it.
docker compose up -d postgres valkey seaweedfs mailpit clamav

# 3. Then the application tier
docker compose up -d app web queue-worker scheduler ffmpeg-worker

# 4. Confirm health before going further
docker compose ps

# 5. Migrate. Already done for you, but harmless to re-run.
docker compose exec app php artisan migrate --force

# 6. Seed roles and reserved usernames (idempotent)
docker compose exec app php artisan db:seed --force

# Demo accounts – only needed if you ever wipe the database
# docker compose exec app php artisan db:seed --class=E2ESeeder --force

# 7. Storage bucket + CORS rules (idempotent; worth re-running –
# your bucket policy predates the Content-Range header additions)
docker compose exec app php artisan media:initialize

# 8. THE FRONTEND – separate terminal, leave it running.
# Vite is deliberately never containerized.
cd web && npm run dev

# 9. Open this, and nothing else:
# https://app.speechcoach.test
```

Picking this up later? Start with these three checks

The stack was left running and fully working on **30 September 2026** – 10 of 11 services healthy, all migrations applied, Vite on :5173. But containers stop when Docker Desktop restarts, and Vite dies when you close its terminal or reboot. So before assuming anything is broken:

```
# 1. Are the containers up? If not, run steps 2–3 above.
docker compose ps

# 2. Is Vite up? If not: cd web && npm run dev
lsof -i :5173

# 3. Any migrations pending? Should print nothing.
docker compose exec app php artisan migrate:status | grep -i pending
```

If all three pass, go straight to §3 and log in. The only container not doing useful work is `whisper-worker`, and nothing in the core flow needs it.

2.3 Host prerequisites – all currently passing

REQUIREMENT	CHECK	STATUS
Hosts entries	<code>grep speechcoach /etc/hosts</code>	PASS both hostnames → 127.0.0.1
TLS certificates	<code>ls docker/certs/</code>	PASS valid until Nov 2028

REQUIREMENT	CHECK	STATUS
mkcert CA trusted	<code>security find-certificate -c mkcert</code>	PASS in System keychain
Frontend env	<code>cat web/.env</code>	PASS <code>VITE_API_URL=https://api.speechco</code> <code>ach.test</code>
Node + deps	<code>node -v</code> ; <code>ls web/node_modules</code>	PASS v22.18.0, 469 packages
Vite running	<code>lsof -i :5173</code>	PASS already listening

3 Accounts & passwords

Five seeded accounts. Every one was logged in through the real browser auth path during this review, and every one succeeded.

The password is `password`

All five fixture accounts share it. All five are pre-verified, so no email confirmation step is needed, and all five have completed onboarding.

EMAIL	PASSWORD	ROLE	ID	USE THEM FOR
<code>member@e2e.test</code>	<code>password</code>	MEMBER	9004	The client / student. Milo Member. Owns the two speeches worth testing, including the one with real published commentary.
<code>coach@e2e.test</code>	<code>password</code>	COACH	9003	The coach. Cora Coach. Holds the published review on speech 17 and an accepted review on speech 9101.
<code>coach-b@e2e.test</code>	<code>password</code>	COACH	9005	A <i>second</i> coach. Use this one to prove isolation – that one reviewer cannot read another’s commentary.
<code>admin@e2e.test</code>	<code>password</code>	ADMIN	9002	Moderation. Can revoke reviews, suspend users, assign roles, read any commentary. Cannot accept invitations or write commentary.
<code>super-admin@e2e.test</code>	<code>password</code>	SUPER_ADMIN	9001	Admin-portal access. Note: in the API this role is almost entirely inert – see §9.

3.1 Your own accounts have no role

Do not test with these

Six real accounts exist from earlier manual testing – ids 1, 2, 3, 29, 41 and 44, including `mars.kwong.cheung@gmail.com`. **None of them has any role assigned.**

That is by design, not corruption: registration deliberately assigns no role, and the frontend comments say outright that `roles: []` is the normal case for every real user. But it has one consequence that will confuse you – the reviewer directory only lists users holding `member` or `coach`, so **a role-less account is invisible there and nobody can invite it to review anything.**

If you want one of your own accounts to behave like a coach:

```
# Grant a role directly. Accepts an id or an email.  
# Roles: super_admin | admin | coach | member  
docker compose exec app php artisan user:grant-role mars.kwong.cheung@gmail.com coach
```

Note this command *replaces* the user’s role set rather than adding to it, and refuses to proceed if it would leave the system without an administrator.

3.2 The admin portal

The admin portal is a separate Filament application – it is not part of the React SPA and is not linked from its navigation.

URL

`https://api.speechcoach.test/control-panel` – note the hostname is the *API* one, and the path is `/control-panel`, not `/admin`. It is unreachable on `localhost:8080`.

Who may enter

Only `admin` or `super_admin`. Everyone else gets 403.

Extra gate

TOTP two-factor is mandatory. On first visit you will be redirected to an authenticator-enrolment screen. Budget time for this; you cannot skip it.

What is there

Users, Speeches, Reports, Coach applications, Connections.

4 How the product works

The design you originally set out, confirmed against the code that was actually built.

A speech is private by default. The *only* thing that grants anyone else access to it is an invitation – and an invitation is not a separate object, it is a row in the `reviews` table.

4.1 The access model

There is no visibility flag on a speech. There are no shares, no links, no public pool. A reviewer can reach a speech if and only if a `reviews` row exists naming them, it has not been revoked, and its status is one of three *access-granting* values:

```
ACCESS_GRANTING = [ 'accepted', 'in_progress', 'published' ]
```

That single rule produces three tiers of what a person sees:

WHO	WHAT THEY GET BACK
Owner	Everything – the full payload and a signed playback URL.
Invited, not yet accepted	Title, duration and the speaker's name only. <i>No playback URL</i> – they cannot watch until they accept.
Everyone else	404, never 403. The system refuses to confirm the speech exists.

Two deliberate absences worth knowing

There is **no endpoint that lists speeches available to review**. This is intentional – with no open pool, the reviewer directory is the only discovery mechanism, which is why it was built as a real searchable feature rather than a list. And **an administrator cannot accept an invitation**; that is refused categorically, not merely unauthorized.

4.2 The invitation loop

1 The speaker invites by name

They open their speech, search the reviewer directory, pick a person, write them a message, and send. A `reviews` row is created with status `invited`. Re-inviting someone who declined reuses the same row rather than failing; inviting yourself is refused outright.

2 The reviewer accepts

The invitation appears in their dashboard. Accepting moves the row to `accepted` – and only now does the playback URL become available to them.

3 The reviewer writes

The first annotation flips the row to `in_progress` automatically.

4 The reviewer publishes

Status becomes `published` and the drafts become visible to the speaker – all at once.

The reviewer may also decline or abandon; the speaker may withdraw, revoke, or revoke-and-purge.

4.3 The draft / publish rule – verified

Drafts are never visible to the client. This was proven, not assumed.

During this review the loop was driven over HTTP against the live app:

ACTION	RESULT
Coach creates a note at t=5s	201 Created
Speaker reads the coach's track <i>while it is a draft</i>	200 – 0 notes
Coach presses Publish	200
Speaker reads the same track again	200 – 1 note

4.4 The four kinds of commentary

KIND	PUBLISH RULE	NOTES
Timed text note	Draft until the review is published; all drafts publish together	The core feature. Timestamp stamps itself on your first keystroke – there is no button. Capped at 200 per review, each up to 120 s long.
Voice note	Same rows, same publish action	Recorded in the browser. Requires the <code>coach</code> role – this is the only thing the coach role actually gates – plus a verified email. Audio up to 16 MB.
Essay	Its own separate Publish	Long-form rich text under the player. Publishing is one-way – the button disables afterwards. Before publication the speaker sees an honest empty state, not an error.
Captions / transcript	No draft lifecycle at all	Not reviewer commentary – these belong to the speech. Only the owner may edit them. Currently non-functional (\$10).

4.5 Coach vs. member – a correction worth noting

“Coach” gates far less than you might expect

A plain `member` can be invited to review, can accept, can write timed notes, can write the essay, and can publish. The `coach` role gates **voice annotations only**.

This is deliberate – the original design called for the peer-review path (member reviewing member) to be a first-class case, precisely so that a stray role check would not silently break it. Worth remembering when you test: if you want to prove peer review works, invite `member@e2e.test` as a reviewer of someone else's speech.

5 Test script A – the coach

Review a client's speech and leave commentary on it. Roughly 10 minutes.

Before you start any script

Confirm the stack is up (`docker compose ps`), Vite is running (`cd web && npm run dev`), and open `https://app.speechcoach.test` . Log out between scripts, or use a private window per persona – the session cookie is shared across the whole `.speechcoach.test` domain.

What already exists, so you don't have to build it

The database is already seeded with a complete, working example. Script A uses it, which is why it needs no setup.

FIXTURE	WHO	STATE
Speech 17	Owned by <code>member@e2e.test</code>	Real 65.7 s video, fully transcoded, plays . Reviewed by <code>coach@e2e.test</code> , who has published 3 timed notes at 14 s, 62 s and 64 s.
Speech 9101	Owned by <code>member@e2e.test</code>	Two coaches hold accepted reviews. Useful for access-control testing, but the video will not play – the file is a deliberate 1 KB placeholder.
Speech 15	Owned by a role-less account	<code>coach-b</code> is invited but has not accepted. Good for testing the pre-acceptance tier.

The walkthrough

1 Log in as the coach

Go to `https://app.speechcoach.test/login` and sign in as `coach@e2e.test` / `password` . *Behind it:* `GET /sanctum/csrf-cookie` then `POST /login` .

2 Open your review dashboard

Navigate to `/dashboard` . You should see your reviews in four sections, oldest invitations first. *Behind it:* `GET /api/reviews` .

3 Go to the speech – by typing the URL

Enter `https://app.speechcoach.test/speeches/17` directly.

Known gap – the dashboard is a dead end

The dashboard renders no link to the speech. There is no anchor, no button, no row click – the speech id is present in the API payload but never rendered as a link. **You must type the URL by hand.** This is a real missing feature, not a configuration problem.

4 Watch the video

Press play. The video should stream and scrub normally. *Behind it:* `GET /api/speeches/17` , then `GET /api/speeches/17/assets/19/playback-url` which returns a presigned SeaweedFS URL. *Verified:* this returns `206 Partial Content` , `video/mp4` , 19,356,216 bytes.

5 Type a note at a timestamp

Pause somewhere and start typing in the notes panel. **The timestamp stamps itself on your first keystroke** – there is no “add note” button to press first. Nudge the time backward and forward in half-second steps using the arrow controls beside the timestamp, and set how long the note should stay on screen. *Behind it:* `POST /api/speeches/17/annotations`.

6 Watch the live preview

Your draft appears over the video with a dashed border marking it provisional – the same overlay component the speaker will eventually see.

7 Add two more, then delete one

Deleting raises a six-second Undo toast. Press it; the note returns.

8 Write the essay

Switch to the Essay tab and write a few paragraphs. It autosaves as you type. *Behind it:* `PUT /api/speeches/17/essay`. **Note the essay has its own separate Publish button** – publishing your notes does not publish the essay.

9 Publish

Press Publish in the notes panel. It reports a count. *Behind it:* `POST /api/reviews/2/publish`. *Verified:* a coach can keep adding and re-publishing notes after an initial publish – the review stays writable.

Optional – prove reviewer isolation

Log in as `coach-b@e2e.test` and try to reach the other coach’s work on speech 17. *Verified:* the API returns **403**, and nothing anywhere reveals that the other review exists.

What will not work in this script

Voice notes will record and upload, but the transcript will fail – the Whisper model file is absent (§10). The audio itself still plays back. **Captions** will not generate for the same reason.

6 Test script B – the client

Upload a speech and get someone to comment on it. Roughly 15 minutes, most of it waiting for the transcode.

Have a video file ready

Any `.mp4` or `.mov`. Keep it short – 30–60 seconds – because transcoding is software-only on macOS (Docker cannot pass through hardware encoding) and runs at roughly real time. The quota is 5 GB per user, with a maximum of two uploads in flight at once.

1 Log in as the client

`member@e2e.test` / `password`. This is Milo Member, your student/client persona.

2 Start a new speech

Go to `/speeches/new`. Give it a title; description and delivery date are optional. Submit. *Behind it:* `POST /api/speeches` creates the record *before* any file is attached.

If you already own a speech you will also see a “this replaces an earlier attempt” picker – choosing one requires a change note.

3 Drop the video in

Drag your file onto the upload panel. It uploads in parts directly to object storage, not through the API. *Behind it:* three calls – open a multipart upload, sign each part, then complete. The browser PUTs the bytes straight to SeaweedFS on :8333. Killing your network mid-upload and restoring it will resume; **reloading the page will not**.

4 Watch it process

Go to `/speeches`. The card polls every three seconds and moves through *Uploading* → *Processing* → *Ready*. You may see an “N videos ahead of yours” hint.

Expect no thumbnail

Poster-frame generation has a history of failing in this environment and zero posters exist in the database. Every card shows a placeholder. The video itself is unaffected – posters are explicitly best-effort and never fail a transcode.

5 Invite a reviewer

Open the finished speech and click **Invite a reviewer** (only the owner sees this). Search the directory and pick **Cora Coach**. Write a message and send. *Behind it:* `GET /api/reviewers` then `POST /api/speeches/{id}/reviews`.

The directory lists only accounts holding `member` or `coach`. Administrators are excluded by design, and so are your own role-less accounts.

6 Confirm the invitation was delivered

Open Mailpit at `http://localhost:8025` to read the invitation email. The in-app notification bell is a separate, database-backed path and is more reliable than the mail.

7**Accept as the coach**

Log in as `coach@e2e.test`, go to `/dashboard`, find the invitation and accept it. *Behind it:* `POST /api/reviews/{id}/accept`. **Only now** does the coach get a playback URL – before accepting they could see only the title, duration and your name.

8**Leave commentary and publish**

Follow steps 4–9 of Script A against your new speech.

Prove the privacy claim

While logged in as `coach-b@e2e.test` – who was never invited – type your new speech’s URL directly. You should get a **404, not a 403**: the system declines to confirm the speech even exists.

7 Test script C – the read-back

The client returns later and reads what the coach wrote. Two minutes, and it needs no setup at all.

This is the path that was broken, and is now fixed

This exact request was returning HTTP 500 before the migration fix in §1.2. It now returns 200 with the coach's three notes – verified directly against the running app.

1 Log in as the client

`member@e2e.test` / `password`.

2 Open the reviewed speech

Go to `https://app.speechcoach.test/speeches/17`. Behind it: `GET /api/speeches/17`.

3 Choose whose commentary to see

In the Notes tab you get a radio group – one option per reviewer who holds an access-granting review, plus **"No commentary"** as a real, selectable option. Pick **Cora Coach**. Behind it: `GET /api/speeches/17/reviews`, then `GET /api/speeches/17/annotations?review_id=2`.

4 Play the video and watch the notes arrive

Three notes should fade in and out on time, at **14 s**, **62 s** and **64 s**. They are tagged by kind – *praise*, *observation*, *correction*.

The two at 62 s and 64 s overlap deliberately – that is the overlay stacking behaviour being exercised.

5 Check the Essay tab

For speech 17 the coach has not published an essay, so you should see an honest empty state rather than an error. That is correct behaviour.

The assertion that matters most

The client sees published work only – never drafts

To prove it yourself rather than taking it on faith: as `coach@e2e.test`, add a fourth note to speech 17 but **do not publish**. Then log back in as `member@e2e.test` and reload. You will still see exactly three notes. Publish as the coach, reload as the client, and the fourth appears.

This was run during the review and behaved exactly so: 0 notes visible while a draft, 1 after publishing.

What the client can see at each stage

REVIEW STATUS	WHAT THE SPEAKER GETS
<code>invited</code>	Reviewer is not offered in the track selector at all.
<code>accepted</code> / <code>in_progress</code>	Reviewer is offered – with an honest "hasn't left commentary yet" empty state. Published notes, if any, are shown; drafts never are.
<code>published</code>	All published notes, plus the essay if it was separately published.

REVIEW STATUS	WHAT THE SPEAKER GETS
revoked	Revocation withdraws the <i>coach's</i> access. The speaker keeps whatever was already delivered to them.

8 Route reference

104 backend routes and 22 SPA routes, taken from `route:list` inside the running container and from the router source – and confirmed identical to the code on disk.

8.1 Authentication – root-mounted, not under `/api`

These are Fortify's own routes and sit at the root deliberately. Note that `/login`, `/register` and `/forgot-password` are *both* SPA pages and API endpoints – nginx disambiguates by HTTP method, so a GET serves the page and a POST reaches Laravel.

METHOD	PATH	PURPOSE
GET	<code>/sanctum/csrf-cookie</code>	CSRF bootstrap. Must be called before any state-changing request.
POST	<code>/register</code>	Create an account. Assigns no role.
POST	<code>/login</code>	Sign in. Throttled to 5 attempts.
POST	<code>/logout</code>	Sign out.
POST	<code>/forgot-password</code>	Send a reset link (read it in Mailpit).
POST	<code>/reset-password</code>	Complete the reset.
GET	<code>/email/verify/{id}/{hash}</code>	The emailed verification link. Signed.
POST	<code>/email/verification-notification</code>	Resend verification.
PUT	<code>/user/password</code>	Change password while signed in.

8.2 Profile & account

METHOD	PATH	PURPOSE
GET	<code>/api/me</code>	Session probe. Returns the user, roles and onboarding state.
GET	<code>/api/onboarding</code>	Onboarding progress.
POST	<code>/api/onboarding/step-1 2 3</code>	The three onboarding steps.
PATCH	<code>/api/profile</code>	Update profile fields.
PATCH	<code>/api/profile/username</code>	Change username (rate-limited by history).
POST	<code>/api/avatar</code>	Upload an avatar.
GET	<code>/api/u/{username}</code>	Public – the only authenticated-free data route.
GET	<code>/api/u/{username}/timeline</code>	Shared review history with that person.
DELETE	<code>/api/account</code>	Erase your account.

8.3 Speeches & media

METHOD	PATH	PURPOSE
GET	<code>/api/speeches</code>	Your own speeches only.
POST	<code>/api/speeches</code>	Create the record, before any file.
GET	<code>/api/speeches/search</code>	Search across captions/transcripts.

METHOD	PATH	PURPOSE
GET	/api/speeches/{id}	One speech. Three access tiers; 404 if you have none.
POST	.../assets/uploads	Open a multipart upload.
POST	.../uploads/{uid}/parts/{n}	Sign one part.
POST	.../uploads/{uid}/complete	Finish; dispatches transcode + captions.
DELETE	.../uploads/{uid}	Abort an upload.
POST	.../assets/{a}/retry	Retry a failed transcode.
GET	.../assets/{a}/playback-url	Presigned playback URL.
GET	/api/queue/transcode-depth	"N videos ahead of yours".

8.4 Reviews – the invitation lifecycle

METHOD	PATH	PURPOSE
GET	/api/reviewers	The reviewer directory. <code>member</code> and <code>coach</code> only.
POST	/api/speeches/{id}/reviews	Invite someone. Owner only.
GET	/api/speeches/{id}/reviews	Who is reviewing this speech.
GET	/api/reviews	Your reviews, in four dashboard sections.
POST	/api/reviews/{id}/accept	Accept. Admins are refused categorically.
POST	/api/reviews/{id}/decline	Decline.
POST	/api/reviews/{id}/withdraw	Reviewer steps back.
POST	/api/reviews/{id}/abandon	Reviewer gives up mid-review.
POST	/api/reviews/{id}/revoke	Speaker (or admin) withdraws access.
POST	/api/reviews/{id}/revoke-and-purge	Revoke and destroy the commentary.
POST	/api/reviews/{id}/publish	Publish all drafts at once.

8.5 Commentary

METHOD	PATH	PURPOSE
GET	.../annotations?review_id=	Read one reviewer's track. <code>review_id</code> is required.
POST	.../annotations	Create a timed note.
PATCH	.../annotations/{id}	Edit. Optimistically locked – 409 on conflict.
DELETE	.../annotations/{id}	Delete (undoable for 6 s in the UI).
DELETE	.../annotation-sets/me	Clear your own set. Takes no author parameter , so no reviewer can target a peer.
POST	.../voice-notes	Upload a voice note. Coach role required.
GET	.../voice-playback-url	Presigned audio URL.
GET/PUT	.../essay	Read / autosave the essay.
POST	.../essay/publish	Separate essay publish. One-way.

METHOD	PATH	PURPOSE
GET/PUT	.../captions	Read / edit captions. Owner edits only.
GET	.../transcript	The transcript.

8.6 Social, privacy & admin

METHOD	PATH	PURPOSE
POST/GET	/api/connections	Request / list connections. Rate-limited.
POST	/api/connections/{id}/accept decline block unblock	Connection lifecycle. Only the blocker can unblock.
GET/POST	/api/notifications, .../{id}/read	The notification bell.
POST	/api/reports	Report a speech.
POST/GET	/api/privacy/exports	Request / list data exports.
POST	/api/coach-applications	Apply to become a coach.
POST	.../{id}/documents	Attach certification PDFs (max 2, 10 MB, ClamAV-scanned).
GET	/control-panel	Filament admin portal. Admin role + TOTP 2FA.
GET	/api/health, /up	Health checks. Unauthenticated.

8.7 Frontend routes

PATH	GUARD	SCREEN
/	NONE	Landing page
/login, /register, /forgot-password	GUEST ONLY	Auth screens; redirect to onboarding if already signed in
/reset-password/:token	NONE	Reset form
/verify	NONE	Verification prompt – deliberately open, for cross-device links
/onboarding	AUTH	Three-step onboarding
/u/:username + /reviews-left, /reviews-received	PUBLIC	Public profile with tabs
/speeches	AUTH + VERIFIED	My speeches, polling for status
/speeches/new	AUTH + VERIFIED	Create + upload
/speeches/:id	AUTH + VERIFIED	The watch / annotate screen
/dashboard	AUTH + VERIFIED	My reviews (no link out – see §10)
/reviewers	AUTH + VERIFIED	Reviewer directory
/search	AUTH + VERIFIED	Search captions/transcripts
/profile, /account	AUTH + VERIFIED	Edit profile; privacy & erasure
/become-a-coach	AUTH + VERIFIED	Coach application

9 Roles & capabilities

Produced by executing the authorization gates per role against real fixture rows – not by reading the policy files and inferring.

There are no “permissions” in this system

The permission tables exist but are **completely empty**, and nothing in the codebase ever references a permission. Authorization is *roles plus policies*, nothing else. Drop “permissions” as a concept when reasoning about this app.

9.1 Who can do what

CAPABILITY	SUPER ADMIN	ADMIN	COACH	MEMBER (SPEAKER)	NO ROLE
Upload a speech	✓	✓	✓	✓	✓
Invite a reviewer to own speech	-	-	-	✓	✓
Appear in the reviewer directory	-	-	✓	✓	-
Accept an invitation	-	-	✓	✓	✓
Write timed notes / essay	-	-	✓	✓	-
Publish a review	-	-	✓	✓	-
Record a voice note	-	-	✓	-	-
Read commentary on a speech	-	✓	✓	✓	-
Edit captions	-	-	-	✓ owner	✓ owner
Revoke / purge a review	-	✓	-	✓ owner	✓ owner
Assign roles, suspend, delete users	-	✓	-	-	-
Enter the admin portal	✓	✓	-	-	-

“owner” means the capability applies to their own speech only.

9.2 Three findings worth knowing

1. `super_admin` is nearly inert in the API

Roles here are mutually exclusive – granting `super_admin` replaces `admin` rather than adding to it, and there is no hierarchy. Almost every policy checks for `admin` specifically, so the super-admin account passes very few of them. Its one real power is entering the admin portal. **For moderation testing, use `admin@e2e.test`, not the super-admin account.**

2. Blocking someone does not restrict their access

The block feature governs *connection requests only*. A blocked user can still be invited as a reviewer, and still reads any speech they already hold a grant on. Connections never widen or narrow what anyone can see – they route a viewer to a profile page, nothing more.

3. An administrator cannot review

Admins are blocked from accepting, declining, withdrawing or abandoning a review, from writing any commentary, and from appearing in or browsing the reviewer directory. They *can* read anyone's commentary and revoke reviews. This is intentional – moderation and participation are kept apart.

9.3 Becoming a coach

Two paths, both ending in the same role assignment:

The product path

`/become-a-coach` → submit an application → attach up to two certification PDFs (magic-byte checked, ClamAV-scanned) → an admin approves it in the portal, which grants the role and writes an audit entry. Rejected applicants may reapply.

The direct path

`php artisan user:grant-role {email} coach` – instant, and what the seeded coaches use. Use this for testing; the application flow is worth exercising separately but is slower.

One gap found: the admin portal has no UI to *grant* a role, only to revoke coach status. Promotion to admin or super-admin is artisan-only.

10 Known broken edges

What will visibly fail, so you can tell a genuine bug from a known gap.

10.1 Will not work today

Captions and voice transcription – the model file is missing

The `whisper-models` volume is **empty**. The worker starts and idles happily, but every captions job fails because there is no model to load. Nothing in the database has a transcript – the transcript table has zero rows.

To fix it (downloads ~142 MiB):

```
docker compose --profile whisper-model-init run --rm whisper-model-init
```

Note that transcription is genuinely slow – it reserves 2 CPUs and 4 GB and pegs them for minutes per job. For a quick functional test you can simply skip captions; nothing else depends on them.

The reviewer dashboard does not link to the speech

After accepting an invitation, a coach has no way to click through to the speech – the dashboard renders no link, button or clickable row anywhere. The speech id is present in the API response but never rendered. Workaround: type `/speeches/{id}` by hand. This is the single most likely thing to make a tester think the app is broken when it is not.

No thumbnails on any speech

Zero poster images exist, and the poster job has a history of failing here. Every card shows a placeholder. Posters are explicitly best-effort and never fail a transcode, so the video itself is fine.

10.2 Rough edges worth knowing

ISSUE	DETAIL
Uploads can get stuck at two	Abandoning a tab mid-upload holds a slot. Two abandoned uploads block the user entirely, and the nightly reconciler is what releases them. Manual unblock: <code>UPDATE users SET uploads_in_flight = 0 WHERE id = 9004;</code>
Reloading mid-upload loses it	Network loss is resumable; a page reload is not – there is no persistence across reloads.
No server-side file-type check	The video type restriction is enforced only in the browser. A direct API call can push any file into storage, where it consumes quota and fails later at the probe step. Worth noting as a real security gap, not just a testing quirk.
Rotated phone video may play sideways	Rotation handling was never exercised against a real fixture. Portrait video from a phone is untested.
A note count reads zero	One seeded review has a stale counter – it shows 0 notes while actually holding 3. Harmless: the authorization path deliberately never reads these counters, precisely so a drifted count cannot deny a speaker commentary they were delivered. Only a profile-timeline count is affected.

ISSUE	DETAIL
Speech 9101's video will not play	Its asset is a deliberate 1 KB placeholder. Fine for access-control testing; use speech 17 when you need a video that actually plays.
Stale mail failures in the queue	Six old failed jobs sit in the queue from August. Clear with <code>php artisan queue:flush</code> .
The spikes page errors	The frontend enables it but the backend flag is unset, so the endpoint returns 403. Cosmetic; unrelated to the product.
<code>README.md</code> is wrong	It still claims there is no application code. It is about fourteen steps out of date – do not use it as a runbook.

11 Troubleshooting

The traps in this stack that cost real hours, each with the symptom that identifies it.

“Something went wrong. Please try again.” on login

Cause: you are on the wrong origin – almost certainly `http://localhost:5173` or `http://localhost:8080`. The backend only accepts `https://app.speechcoach.test`, so the browser blocks the request and the SPA has no error payload to display, which is why the message is generic rather than specific.

Fix: use `https://app.speechcoach.test`. Your credentials were never the problem.

A backend change you just made has no effect

Cause: the `app` container has *no bind mount* – `api/` is copied into the image at build time. The container happily runs the old code and commands succeed against it, which makes this silent and very confusing.

The tell: `migrate` says “Nothing to migrate” while migration files plainly exist on disk. Or a route 404s that you can see in `routes/api.php`.

```
docker compose build app queue-worker scheduler
docker compose up -d app queue-worker scheduler
```

Exception: `api/.env` is *not* baked in – it is injected at container-create time. Env changes need `docker compose up -d --force-recreate app`, not a rebuild.

`https://app.speechcoach.test` returns 502

Cause: the host Vite dev server is not running. nginx proxies that hostname to port 5173 on your Mac, and Vite is deliberately never containerized. **Fix:** `cd web && npm run dev`.

Nothing starts, and `clamav` sits at “health: starting”

Cause: ClamAV is loading virus signatures; its startup grace period is three minutes. Both `app` and `queue-worker` hard-depend on it being healthy, so the whole stack waits. **Fix:** wait it out. This is normal, not a hang.

Pages load slowly or a navigation seems to hang

Cause: you are driving the Vite dev server, which compiles modules on demand, single-threaded. Under host load this can stall for a minute or more. It tracks your Mac’s CPU, not the app. **Fix:** be patient on first load of a route; it is fast afterwards. Known causes already ruled out: dev-server age, stray browser processes, and container load.

Port conflicts on 443, 5433 or 8333

Cause: the end-to-end test stack is running at the same time as the dev stack. They contend for the same ports and must never both be up. **Fix:** `scripts/e2e-stack.sh down` first. A leftover Docker network from a previous run can also block startup with a label conflict – check it has no containers attached before removing it.

Quick diagnostic commands

```
# Is everything healthy?
docker compose ps

# Is the container's code current? (should print 104)
docker compose exec app php artisan route:list --json | python3 -c "import
json,sys;print(len(json.load(sys.stdin)))"

# Any migrations pending? (should say Nothing to migrate)
docker compose exec app php artisan migrate:status | grep -i pending

# Is the API alive?
curl -sk https://api.speechcoach.test/api/health

# Is Vite up?
lsof -i :5173

# Read the mail the app has sent
open http://localhost:8025

# What is actually in the database?
docker compose exec postgres psql -U speechcoach -d speechcoach \
  -c "SELECT id, user_id, title FROM speeches ORDER BY id;"

# Watch a transcode happen
docker compose logs -f ffmpeg-worker
```

Resetting to a clean slate

This destroys your existing data

Only do this if you want the fixtures back from scratch. It will delete the four existing speeches and your six real accounts.

```
docker compose exec app php artisan migrate:fresh --seed --force
docker compose exec app php artisan db:seed --class=E2ESeeder --force
docker compose exec app php artisan media:initialize
```

Two further optional seeders add richer fixtures – caption data and voice annotations with real audio. Both need the storage service healthy:

```
docker compose exec app php artisan db:seed --class=E2ECaptionsSeeder
docker compose exec app php artisan db:seed --class=E2EVoiceAnnotationSeeder
```

Verification basis. Compiled 30 September 2026 against the running stack on this machine. Credentials were confirmed by real logins; routes by `route:list` inside the container; the permission matrix by executing gates per role; the commentary loop by driving it over HTTP; and video playback by a range request returning `206 Partial Content`. Where something could not be verified it is marked as such rather than asserted. The nine pending migrations were applied during this review, after a database dump was taken; the two fixture rows touched while proving the publish loop were restored to their original state.